

GX 사업본부 플러그인 가이드

oh-my-gx 플러그인 가이드

기준 버전 v1.21.1 · 최종 수정 2026-08-05



목차 · 이 문서 읽는 법

1~4장은 설치부터 도메인 지식 등록까지 처음 한 번 거치는 구간이고, 5~7장은 스킬을 부르고 질문에 답하는 방법입니다. 8~15장은 스킬별로 왜 존재하고 언제 쓰는지, 실제로 입력할 문장을 모아 두었습니다.

장	제목	언제 보나
1	무엇을 하는 플러그인인가	도입 검토
2	설치 — 터미널에서 시작	처음 1회
3	초기 설정 — /gx-setup	처음 1회
4	/gx-context — 도메인 지식	처음 1회
5	스킬 부르는 법	처음 1회 (중요)
6	중간에 물어보면	처음 1회 (중요)
7	스킬 17종 지도	무엇을 쓸지 고를 때
8	/gx-dev — 새 기능 개발	전체 과정이 필요할 때
9	/gx-dev — 작은 변경 · 재개	설정값·버그 수정
10	/gx-tdd — 테스트로 증명	로직·버그 수정
11	/gx-lens · /gx-tech-debt	고치기 전 파악
12	/gx-research · /gx-humanizer	조사 · 문서 작업
13	커밋 · PR · 교차 검증	마무리
14	세션이 끊겨도 이어지는 이유	작업이 길어질 때
15	다른 언어 · SVN · 막히는 지점	막혔을 때

<표 0-1> 장별 구성

읽는 순서

1. 처음이라면 1장부터 6장까지 순서대로 읽으세요.
2. 8~13장은 통독하지 말고 그 스킬을 쓸 때 펼쳐 보세요.
3. Java·Node가 아닌 스택은 15장을 먼저 확인하세요.
4. 각 장의 표에 실제로 입력할 문장이 그대로 적혀 있습니다.

1. 무엇을 하는 플러그인인가

개발 요청 하나를 받아 요구사항 정리 → 설계 → 구현 → 리뷰 → 커밋 → PR 생성까지 끌고 가는 Claude Code 플러그인입니다. 단계마다 담당이 나뉘어 있고, 중요한 판단 지점에서는 사람에게 물어봅니다.

왜 단계를 나누나.

한 사람이 만들고 그 사람이 검토하면 자기가 놓친 것을 자기가 찾지 못합니다. 설계자와 비판자, 구현자와 리뷰어를 갈라 놓고 서로 다른 관점에서 보게 만드는 것이 이 플러그인의 핵심입니다.

무엇을 입력하나.

모든 기능은 /gx- 로 시작하는 슬래시 명령으로 부릅니다. 명령 뒤에 하고 싶은 말을 자연어로 붙이면 그것이 곧 옵션입니다 (5장).

읽는 순서.

2장에서 설치하고 3·4장에서 초기 설정과 도메인 지식을 마친 뒤, 5·6장에서 부르는 법과 되묻는 방식을 익히면 나머지는 필요할 때 찾아 읽으면 됩니다.

한다	하지 않는다
모호한 요청을 질문으로 다듬어 문서화	사용자 승인 없이 구현 진입
기술 설계 + 다른 관점의 비판 검토	PR 머지 (사람이 직접)
코드 구현 + 자기점검, 리뷰 2~3관점	강제 푸시 (차단합니다)
테스트 실행 결과를 확인한 뒤 완료 판정	보호 브랜치에 직접 커밋 (차단합니다)
도메인 지식을 문서로 축적·재사용	민감 파일을 조용히 커밋 / SVN 커밋 대행

<표 1-1> 자동화 범위와 사람이 판단하는 영역

승인 게이트

1. 요구사항 승인 — 무엇을 만들지 확정
2. 설계 승인 — 어떻게 만들지 확정
3. 리뷰 결과 확인 — 지적 사항 수용 여부 판단
4. 인수 검증 — 의도대로 됐는지 확인. 머지는 사람이 직접 합니다.

2. 설치 — 터미널에서 시작

1. 터미널에서 프로젝트로 이동합니다.

플러그인은 프로젝트 단위로 붙습니다. 작업할 저장소 루트로 이동한 뒤 `claude` 명령으로 Claude Code를 띄우세요.

2. 플러그인을 설치합니다.

Claude Code 안에서 아래 두 줄을 차례로 입력합니다.

```
/plugin marketplace add bs-koo/oh-my-gx
/plugin install oh-my-gx@oh-my-gx
```

3. 초기 설정을 실행합니다.

설치가 끝나면 `/gx-setup` 을 입력하세요. 이 명령만은 자연어로 호출되지 않으므로 반드시 직접 입력해야 합니다 (3장).

제대로 붙었는지 확인.

`/` 를 눌렀을 때 `gx-` 명령이 목록에 뜨면 정상입니다.

입력	무엇을 하나
<code>claude</code>	터미널에서 Claude Code 실행
<code>/plugin marketplace add bs-koo/oh-my-gx</code>	플러그인 저장소 등록
<code>/plugin install oh-my-gx@oh-my-gx</code>	플러그인 설치
<code>/plugin marketplace update oh-my-gx</code>	최신 버전으로 갱신
<code>/gx-setup</code>	프로젝트 초기 설정 (3장)

<표 2-1> 설치에 쓰는 명령

설치 확인

- 1. Claude Code CLI가 먼저 설치되어 있어야 합니다.
- 2. 플러그인은 프로젝트별로 붙습니다. 새 저장소마다 설치하세요.
- 3. `/plugin` 명령은 터미널 셸이 아니라 Claude Code 안에서 입력합니다.
- 4. 설치되면 `/` 를 누를 때 `gx-` 명령이 목록에 나타납니다.

3. 초기 설정 — /gx-setup

프로젝트마다 버전 관리 도구도, 빌드 명령도, 인증 방식도 다릅니다. /gx-setup은 이것들을 한 번에 확정해 이후 모든 스킴이 같은 값을 쓰게 만듭니다.

왜 직접 입력해야 하나.

"초기 설정 해줘"라고 말해도 실행되지 않도록 막아 두었습니다. 설정을 실수로 덮어쓰는 사고를 막기 위해서입니다.

중간에 물어봅니다.

감지한 값이 맞는지 단계마다 확인합니다. 선택지에 답이 없으면 마지막 항목을 골라 직접 입력하면 됩니다 (6장).

이미 설정된 값은 덮어쓰지 않으므로 다시 실행해도 안전합니다. 설정이 꼬였다 싶을 때 가장 먼저 해볼 조치입니다.

단계	무엇을 하나
0 · VCS 감지	git인지 svn인지 판별해 기록
0.5 · 프로젝트 타입 등록	빌드 파일을 훑어 빌드·테스트 명령을 제안 → 확인
1 · 필수 도구 확인	gh CLI 확인. JDK는 java 계열일 때만
2 · 인증	GitHub 인증 상태 확인
3 · 도메인 지식 안내	context 등록 경로 안내 (4장)
4 · 알림 (선택)	Google Chat 웹훅 등록
5 · 모델 프로파일 (선택)	표준 · 에코 기본값 지정 (9장)

<표 3-1> /gx-setup 7단계

설정 확인

1. gh CLI가 없으면 OS별 설치 명령을 안내합니다.
2. JDK 확인은 java 계열 프로젝트에서만 수행합니다.
3. 설정이 꼬였다 싶으면 /gx-setup 을 다시 실행하면 됩니다.
4. Java·Node가 아닌 스택은 0.5단계가 핵심입니다 (15장).

4. /gx-context — 도메인 지식

왜 존재하나.

Claude는 코드를 읽을 수 있지만 "우리 회사에서 이 단어가 무슨 뜻인지"는 모릅니다. 도메인 용어·구조·관련 저장소를 문서로 등록해 두면 매 요청마다 배경을 설명하지 않아도 됩니다.

폴더는 이렇게 생깁니다.

프로젝트 루트의 context 폴더 아래 도메인별 하위 폴더가 생기고, 그 안에 용어·구조·관련 저장소·진행 상황 문서가 들어갑니다. 도메인이 늘면 폴더가 하나씩 추가됩니다.

팀이 함께 씁니다.

이 폴더는 저장소에 커밋됩니다. 한 사람이 만들면 팀 전체가 같은 지식을 참조하게 되고, 개발 요청마다 관련 도메인 문서를 자동으로 읽어 갑니다.

언제 쓰나.

플러그인을 처음 붙였을 때, 새 도메인이 생겼을 때, 정책이 바뀌어 문서와 코드가 어긋났을 때 씁니다.

경로	무엇이 들어가나
context/glossary.md	프로젝트 공통 용어
context/에너지-분석/	도메인 하나가 폴더 하나
└ glossary.md	그 도메인의 용어 사전
└ architecture.md	구조와 주요 흐름
└ PROJECTS.md	관련 저장소 목록
└ status.md	기능별 구현 상태

<표 4-1> context 폴더 구조

- 이럴 때 이렇게**
1. 처음 붙였을 때 자동 생성 — /gx-context
 2. 도메인 직접 등록 — /gx-context 에너지-분석
 3. 문서에서 등록 — /gx-context 에너지-분석 문서 기반으로 등록해줘
 4. 진행도 반영 — /gx-context 에너지-분석 진행도 동기화해줘

5. 스킬 부르는 법

형식은 하나입니다.

슬래시 명령으로 스킬을 지정하고, 뒤에 하고 싶은 말을 자연어로 붙입니다. 옵션 이름을 외울 필요 없이 문장으로 표현하면 됩니다.

왜 슬래시로 부르나.

그냥 말해도 알아듣는 경우가 있지만 어떤 절차가 돌지 예측할 수 없습니다. 슬래시로 지정하면 의도한 스킬이 정확히 실행되고, 팀원과 같은 방식으로 쓰게 됩니다. 이 문서는 항상 슬래시 호출을 기준으로 설명합니다.

뒤에 붙이는 말이 곧 옵션입니다.

"상세하게", "가볍게", "PRD만", "이어서" 같은 표현을 문장 안에 자연스럽게 섞어 쓰면 그대로 반영됩니다.

이렇게 입력하면	이렇게 동작합니다
/gx-dev 대시보드 추가해줘	전체 개발 사이클 진행
/gx-dev 단위 변경, 가볍게 해줘	질문 단계를 줄여 진행
/gx-dev 설비 분석 기능, PRD만 작성해줘	요구사항 문서만 만들고 종료
/gx-dev 이어서 해줘	멈춘 단계부터 재개
/gx-lens 정산 정책 확인해줘, 상세하게	더 깊이 탐색해 보고서 생성
/gx-context 에너지-분석	해당 도메인 지식 등록·갱신
/gx-commit 알림 설정 화면 추가	지정한 메시지로 커밋

<표 5-1> 입력 형태와 결과

입력 요령

1. / 를 누르면 사용 가능한 gx- 명령이 목록에 나옵니다.
2. 명령만 입력하고 엔터를 쳐도 됩니다 — 필요한 것을 되물어봅니다.
3. 한 번에 한 가지만 요청하세요. 범위가 커지면 되돌아오는 횟수가 늘니다.
4. 표현은 자유입니다. "빨리 고쳐줘"와 "긴급 수정"은 같게 동작합니다.

6. 중간에 물어보면

스킬이 진행 중에 선택지를 띄우는 지점이 있습니다. 진행 방식이나 결과물 형태처럼 사람이 정해야 하는 것들입니다.

선택지에 답이 없으면 직접 쓰면 됩니다.

목록의 마지막 항목은 항상 직접 입력하는 자리입니다. 그 자리를 골라 원하는 바를 문장으로 쓰면 그대로 반영됩니다. "둘 다 아니고 설계만 보고 싶다"처럼 선택지에 없는 요구도 그냥 쓰면 됩니다.

미리 쓰면 묻지 않습니다.

명령에 "가볍게", "에코로"처럼 미리 적어 두면 해당 질문 자체가 생략됩니다.

질문이 한 창에 여러 개 뜨기도 합니다. 진행 방식과 모델을 한꺼번에 고르는 경우가 그렇습니다.

답을 바꾸고 싶으면.

이미 고른 뒤라도 다음 대화에서 말하면 그때부터 반영됩니다.

언제	무엇을 묻나	미리 쓰면 생략
/gx-dev 시작	전체 / 핵심 진행 방식	"가볍게", "전체 과정"
/gx-dev 시작	표준 / 에코 모델	"에코로"
/gx-dev 구현 진입	대화형 / 무인 루프	—
/gx-lens 보고서 후	영향도 분석 여부	변경안을 함께 입력
/gx-research 시작	조사 깊이 · 결과 형태	"요약만", "비교 분석"
/gx-humanizer 시작	점검만 / 고치기 / 교차검증	"감사만", "고쳐줘"
/gx-context 기존 도메인	무엇을 갱신할지	갱신 내용을 함께 입력

<표 6-1> 물어보는 지점

답하는 요령

1. 선택지가 애매하면 마지막 항목에 이유를 적어 주세요.
2. "잘 모르겠으니 추천해줘"라고 써도 됩니다.
3. 방향이 틀렸으면 다음 질문에서 바로잡아도 됩니다.
4. 자주 쓰는 선택은 /gx-setup 에서 기본값으로 고정할 수 있습니다.

7. 스킬 17종 지도

17종을 전부 외울 필요는 없습니다. 처음에는 /gx-setup, /gx-context, /gx-dev 세 개면 충분하고 나머지는 필요할 때 찾아 쓰면 됩니다.

고르는 기준.

코드를 고치는 스킬과 읽기만 하는 스킬이 나뉩니다. 무엇을 건드릴지 모르겠으면 읽기 전용부터 돌려 현황을 파악한 뒤 개발 스킬로 넘어가는 순서가 안전합니다.

표의 괄호 안 장 번호를 따라가면 그 스킬의 상세 설명이 나옵니다.

구분	스킬	역할
설치·기반	/gx-setup	초기 설정. 직접 입력해야 함 (3장)
	/gx-context	도메인 지식 등록·갱신 (4장)
개발	/gx-dev	전체 개발 사이클 (8·9장)
	/gx-tdd	테스트를 먼저 쓰고 통과 확인 (10장)
	/gx-ralph	1건씩 무인 반복 (SVN 미지원)
TDD 단계	/gx-red · green · refactor · verify	한 단계씩 손으로 돌릴 때
읽기 전용	/gx-lens	현행 정책 탐지 + 변경 영향도
	/gx-tech-debt	코드·구조·의존성·테스트 부채 분석
	/gx-research	웹 검색 리서치 (출처 URL 명시)
커밋·검증	/gx-commit · pull-request · cross-review	커밋·PR 생성, 산출물 교차 검증
글쓰기	/gx-humanizer	AI 글쓰기 흔적 교정

<표 7-1> 스킬 분류와 진입점

처음 세 개

1. /gx-setup — 프로젝트에 붙일 때 한 번
2. /gx-context — 도메인 지식을 등록할 때
3. /gx-dev — 매 작업마다
4. 나머지는 이름만 기억해 두고 필요할 때 이 표를 펼치세요.

8. /gx-dev — 새 기능 개발

왜 존재하나.

요청 하나를 받아 요구사항 정리부터 PR까지 끌고 갑니다. 혼자 만들고 혼자 검토하면 놓친 것을 못 찾기 때문에, 단계마다 다른 관점의 담당을 붙이고 사람이 승인할 지점을 만들어 두었습니다.

네 번 멈춥니다.

요구사항 승인, 설계 승인, 리뷰 지적 확인, 인수 검증 보고. 그 사이는 알아서 진행합니다.

언제 쓰나.

새 기능처럼 무엇을 만들지부터 정리해야 하는 작업에 씁니다. 설정값만 바꾸는 일이라면 다음 장의 핵심 모드가 낫습니다.

무인 루프로 넘기려면.

구현 단계에 들어갈 때 대화형으로 갈지 무인 루프로 돌릴지 한 번 물어봅니다. 수용 기준이 여러 개일 때 1건씩 자동으로 처리하고, 리뷰와 PR은 돌아온 뒤에 진행합니다.

이럴 때	이렇게 입력하세요
새 기능 개발	/gx-dev 건물별 에너지 사용량 대시보드 추가해줘
이슈 키가 있을 때	/gx-dev [ENG-245] 전력 수요 예측 API 추가
요구사항만 먼저	/gx-dev 설비 가동률 분석 기능, PRD만 작성해줘
설계만	/gx-dev 설계만 해줘
리뷰만	/gx-dev 리뷰만 해줘
다른 브랜치에서 분기	/gx-dev develop에서 월간 리포트 기능 추가해줘

<표 8-1> 이럴 때 이렇게 입력하세요

진행 중에

1. 요구사항·설계는 승인해야 다음으로 넘어갑니다.
2. 이슈 키를 대괄호로 넣으면 브랜치 이름에 반영됩니다.
3. 질문이 많다 싶으면 "가볍게 해줘"로 범위를 줄이세요 (9장).
4. 중간에 멈춰도 다음 세션에서 이어집니다 (14장).

9. /gx-dev — 작은 변경 · 재개

핵심 모드 — 작은 변경.

설정값 변경이나 문구 수정에 요구사항 문서와 설계까지 만들면 배보다 배꼽이 큼니다. 확인 → 구현 → 검증으로 줄이되 기록과 검증은 그대로 남깁니다.

에코 — 토큰 절약.

절차는 그대로 두고 모델만 낮춥니다. Pro 요금제에서 상시 쓰기 좋고, /gx-setup 에서 기본값으로 고정할 수 있습니다.

재개 — 중단한 작업.

세션을 닫아도 진행 상태가 남습니다. 걸려 있는 작업이 여러 개면 어느 것을 이어갈지 물어봅니다 (14장).

줄여도 남는 것.

수용 기준과 변경 요약이 남고, 검증을 통과해야 커밋으로 갑니다.

이럴 때	이렇게 입력하세요
설정값 · 문구 변경	/gx-dev 표시 단위를 kWh에서 MWh로 변경해줘
처음부터 가볍게	/gx-dev 임계값을 800kW로 변경, 가볍게 해줘
긴급 버그 수정	/gx-dev 센서 데이터 수집 오류 긴급 수정해줘
토큰 절약	/gx-dev --eco 알림 재시도 로직 추가해줘
중단한 작업 재개	/gx-dev 이어서 해줘
진행 상태 확인	/gx-dev 지금 어디까지 됐어?

<표 9-1> 이럴 때 이렇게 입력하세요

알아둘 것

1. "긴급", "핫픽스", "빨리"가 들어가면 핵심 모드로 판정합니다.
2. 핵심 모드도 기록과 빌드·테스트 검증은 그대로 남습니다.
3. 에코는 검증 기준을 낮추지 않습니다. 모델만 내려갑니다.
4. 재개는 브랜치별로 관리되므로 브랜치를 확인하고 부르세요.

+ 10. /gx-tdd — 테스트로 증명

왜 존재하나.

"다 됐습니다"를 믿으려면 근거가 있어야 합니다. 테스트를 먼저 쓰고 실제로 돌려 통과를 확인하기 전에는 커밋으로 넘어가지 않습니다. 테스트를 나중에 쓰면 코드에 맞춰 쓰게 되므로 순서를 강제합니다.

무엇이 달라지나.

구현이 실패 테스트 → 최소 구현 → 정리 순환으로 바뀌고, 리뷰가 요구사항 충족과 코드 품질 두 단계로 나뉩니다.

언제 쓰나.

정답을 자동 테스트로 표현할 수 있으면 씁니다. 계산·검증·결제·인증 로직과 버그 수정이 대표적입니다.

한 단계씩 돌리기.

전체를 돌리지 않고 실패 테스트만 쓰거나 통과만 시킬 수도 있습니다. 단계별 스킴을 따로 부르면 되고, 손으로 한 사이클만 돌려 보고 싶을 때 유용합니다.

이럴 때	이렇게 입력하세요
계산 · 검증 로직 개발	/gx-tdd 전력 요금 정산 로직 구현해줘
버그 수정	/gx-tdd 센서 값 누락 시 오동작하는 버그 고쳐줘
한 단계씩 직접	/gx-red → /gx-green → /gx-refactor
완료 검증만	/gx-verify
검증에 막혔을 때	고친 뒤 /gx-verify 다시 실행

<표 10-1> 이럴 때 이렇게 입력하세요

검증 게이트

1. 테스트가 하나라도 실패하면 커밋으로 넘어가지 않습니다.
2. 실패한 테스트 이름이 함께 나오므로 그것부터 고치세요.
3. 테스트가 PC에서 실행 가능해야 이 사이클이 성립합니다.
4. 건너뛰려면 명시해야 하고, 그 사실이 기록에 남습니다.

11. /gx-lens · /gx-tech-debt

왜 존재하나.

고치기 전에 현행을 알아야 합니다. 정책이 코드 여기저기 흩어져 있으면 무엇을 건드릴지 판단할 수 없습니다. 두 스킴 모두 코드를 고치지 않고 현황만 보고서로 돌려줍니다.

/gx-lens — 정책과 영향도.

지금 어떻게 동작하는지 위치를 찾아 정리하고, 변경안을 함께 주면 영향 범위까지 판정합니다.

/gx-tech-debt — 부채 우선순위.

코드·구조·의존성·테스트 네 유형으로 나눠 분석하고 무엇부터 손댈지 로드맵을 냅니다.

언제 쓰나.

인수인계를 받았을 때, 오래된 코드를 손대기 전에, 변경 범위를 산정해야 할 때 먼저 돌려 보세요. 보고서는 문서로 저장되어 팀에 공유할 수 있습니다.

이럴 때	이렇게 입력하세요
현행 정책 확인	/gx-lens 에너지 요금 정산 정책 확인해줘
더 깊이 파악	/gx-lens 전력 피크 알림 정책 확인해줘, 상세하게
변경 영향도까지	/gx-lens 가동률 정책 확인해줘, 70%로 낮추려고 해
기술 부채 파악	/gx-tech-debt
특정 영역만	/gx-tech-debt 테스트 커버리지 위주로 봐줘

<표 11-1> 이럴 때 이렇게 입력하세요

활용

1. 인수인계 받을 때 도메인별로 한 번씩 돌려 보세요.
2. 변경안을 같이 주면 보고서와 영향도를 한 번에 받습니다.
3. 영향도는 바로 진행 / 추가 검토 / 신중히 중 하나로 나옵니다.
4. 코드를 고치지 않으므로 안심하고 돌려도 됩니다.

12. /gx-research · /gx-humanizer

/gx-research — 외부 조사.

웹 검색과 문서 분석을 병행해 조사 결과를 냅니다. 모든 발견에 출처 URL을 남기므로 근거를 다시 확인할 수 있습니다.

조사 결과를 지식으로 옮기려면.

리서치가 끝나면 그 결과를 도메인 지식으로 옮기는 명령을 안내합니다. 그대로 실행하면 이후 개발에서 기술 선택 근거로 참조됩니다.

/gx-humanizer — 글 교정.

AI가 쓴 티가 나는 문장을 자연스럽게 고칩니다. 점검만 할지, 고칠지, 교차 검증까지 할지 고를 수 있습니다.

언제 쓰나.

기술 선택 근거나 시장 동향이 필요할 때 조사 스킬을 씁니다.

이럴 때	이렇게 입력하세요
기술 트렌드 조사	/gx-research 스마트그리드 에너지 관리 트렌드
기술 비교	/gx-research LSTM vs Prophet 비교, 비교 분석으로
빠른 요약	/gx-research 탄소 배출권 거래제 현황, 요약만
문서 점검만	/gx-humanizer docs/energy-blog.md
텍스트를 바로 교정	/gx-humanizer 뒤에 텍스트를 붙여넣기

<표 12-1> 이럴 때 이렇게 입력하세요

알아둘 것

1. 리서치 결과는 개인 조사물이라 저장소에 커밋되지 않습니다.
2. 조사 깊이와 결과 형태를 시작할 때 물어봅니다 (6장).
3. 교정은 점검만 / 고치기 / 교차검증 중에서 고릅니다.
4. 파일 경로 대신 텍스트를 그대로 붙여넣어도 동작합니다.

13. 커밋 · PR · 교차 검증

/gx-commit — 커밋 메시지.

브랜치 이름에서 타입을 읽고 변경 내용을 분석해 한국어 커밋 메시지를 만듭니다. 컨벤션을 매번 신경 쓰지 않아도 됩니다.

/gx-pull-request — PR 본문.

커밋 히스토리를 읽어 제목과 본문(배경·요약·변경점·체크리스트)을 채웁니다. 머지는 사람이 합니다.

/gx-cross-review — 약속 대비 점검.

개발이 끝난 뒤 요구사항·설계 문서와 실제 결과물을 대조합니다. 일반 코드 리뷰와 보는 각도가 다릅니다.

따로 부르지 않아도 되는 경우.

개발 스킬로 끝까지 진행했다면 마지막 단계에서 커밋과 PR이 이어서 실행됩니다. 중간에 멈췄거나 직접 고친 경우에만 따로 부르면 됩니다.

이럴 때	이렇게 입력하세요
변경사항 커밋	/gx-commit
메시지를 직접 지정	/gx-commit 전력 피크 알림 설정 화면 추가
PR 생성	/gx-pull-request
베이스 브랜치 지정	/gx-pull-request develop
커밋과 PR 연달아	/gx-commit 실행 후 /gx-pull-request
산출물 교차 검증	/gx-cross-review

<표 13-1> 이럴 때 이렇게 입력하세요

주의

1. 보호 브랜치에서는 커밋이 차단됩니다. 작업 브랜치를 먼저 만드세요.
2. SVN 프로젝트는 두 스킬을 지원하지 않습니다 (15장).
3. PR은 생성까지입니다. 머지는 사람이 직접 합니다.
4. 민감 파일이 잡히면 커밋 전에 경고합니다.

14. 세션이 끊겨도 이어지는 이유

진행 상태가 파일로 남습니다.

작업을 시작하면 브랜치마다 전용 폴더가 만들어지고, 어느 단계까지 왔는지가 state.md에 기록됩니다. 세션을 닫든 컴퓨터를 켜든 이 파일이 남아 있어 "이어서 해줘"라고 하면 그 지점부터 다시 시작합니다.

다시 묻지 않습니다.

승인한 요구사항과 설계도 같은 폴더에 남습니다. 새 세션에서 이어가도 처음부터 다시 설명할 필요가 없습니다.

팀과 공유됩니다.

이 폴더는 저장소에 커밋됩니다. 다른 사람이 이어받을 때 무엇을 왜 만들기로 했는지, 리뷰에서 무엇이 지적됐는지 그대로 볼 수 있습니다.

파일	무엇이 적혀 있나
state.md	현재 단계와 진행 상태 — "이어서"가 읽는 파일
prd.md	요구사항 문서 (승인본)
design.md	설계 문서 (승인본)
codemap.md	이 작업과 관련된 코드 위치
self-check.md	구현 자기점검 결과
trust-ledger.md	보안 감사 기록
ac.md · summary.md	작은 변경의 기준과 결과 요약

<표 14-1> 작업 브랜치 폴더에 남는 것

재개하는 법

1. "/gx-dev 이어서 해줘" — 멈춘 단계부터 재개
2. "/gx-dev 지금 어디까지 됐어?" — 현재 단계 확인
3. 걸려 있는 작업이 여러 개면 어느 것을 이어갈지 물어봅니다.
4. 개인 조사 결과는 공유 대상이 아니라 커밋되지 않습니다.

+ 15. 다른 언어 · SVN · 막히는 지점

다른 언어 · 프레임워크.

Java와 Node 외의 스택은 /gx-setup 이 빌드 파일을 훑어 빌드·테스트 명령을 제안합니다. 맞다고 답하면 등록되고 이후 모든 검증이 그 명령을 씁니다. 등록하지 않으면 검증 단계에서 막힙니다.

SVN 프로젝트.

/gx-commit 과 /gx-pull-request 는 지원하지 않습니다. 리뷰까지 진행한 뒤 터미널에서 직접 커밋하세요. 나머지 스킴은 동일하게 동작합니다.

막히면 아래 표를 보세요.

표에 없는 증상은 그대로 물어보세요. 오류 메시지를 붙여넣으면 원인을 짚어 줍니다. 설정이 꼬였다 싶을 때는 /gx-setup 재실행이 가장 빠른 해결책입니다.

증상	조치
"초기 설정 해줘"가 안 먹힘	/gx-setup 을 직접 입력
커밋이 차단됨	보호 브랜치입니다. 작업 브랜치를 만든 뒤 재시도
빌드·테스트 명령을 못 찾음	/gx-setup 재실행으로 등록
검증 미통과로 커밋이 막힘	실패 원인을 고친 뒤 /gx-verify
PR 생성이 권한 오류로 실패	gh 인증 상태 확인 후 계정 전환
중단한 작업을 다시 시작	/gx-dev 이어서 해줘 (14장)
토큰 소모가 크게 느껴짐	--eco 또는 "가볍게"를 붙여 실행

<표 15-1> 증상과 조치

스택별 참고

1. Makefile · CMakeLists.txt · pyproject.toml 등을 자동으로 인식합니다.
2. 표에 없는 스택도 빌드 스크립트를 근거로 명령을 제안합니다.
3. 임베디드는 테스트가 PC에서 돌아야 TDD 사이클이 성립합니다.
4. SVN은 브랜치 없이 trunk에서 작업합니다.